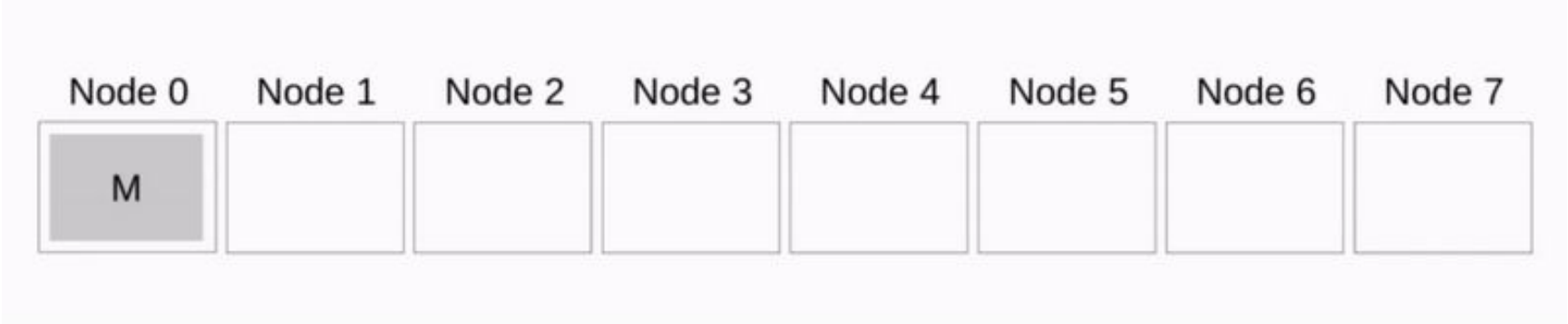
Collective Communication

Broadcast in MPI — Why Multiple Methods?

- MPI does **not use a single fixed broadcast algorithm**
- Multiple broadcast methods exist with different trade-offs
- MPI **selects the broadcast method internally** before communication starts
- Selection depends on message size, process count, node count, and network
- Different methods optimize for latency, bandwidth, or scalability
- Understanding these methods explains why MPI_Bcast performance varies

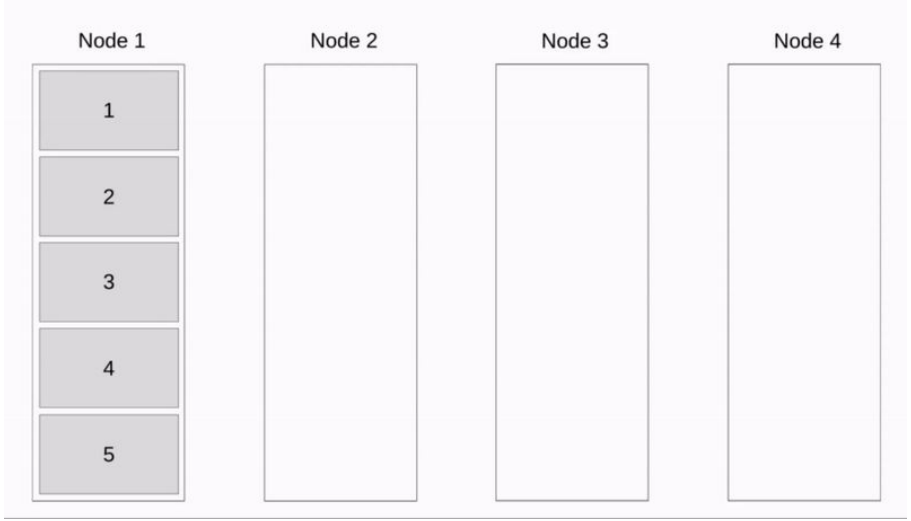
Binomial Tree Broadcast

- A **binomial tree broadcast** is a broadcast algorithm in which the data is distributed by repeatedly doubling the number of processes that hold the message at each communication step.
- Initially only the root sends the message, and in each subsequent time step the number of sending **nodes doubles**, resulting in exponential growth of active senders.
- Only **one node is active** during the first transfer, leading to poor bandwidth utilization and idle processes.



Linear Pipeline Broadcast

- Pipeline broadcast splits a long message into **smaller chunks** and forwards these chunks **as soon as they are received**, instead of waiting for the entire message.
- The root divides the message into multiple segments, and each process receives a chunk and immediately forwards it to the next process, allowing sending and receiving to overlap in time.
- ***The runtime depends on both the message length and the number of processors.***



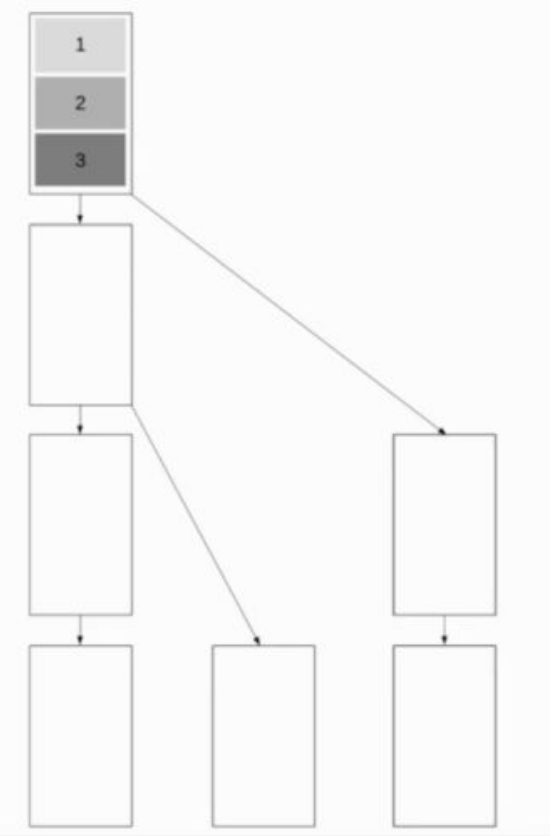
Pipelined Binary Tree Broadcast

- This algorithm combines the advantages of:

  Binomial Tree Broadcast

  Linear Pipeline Broadcast

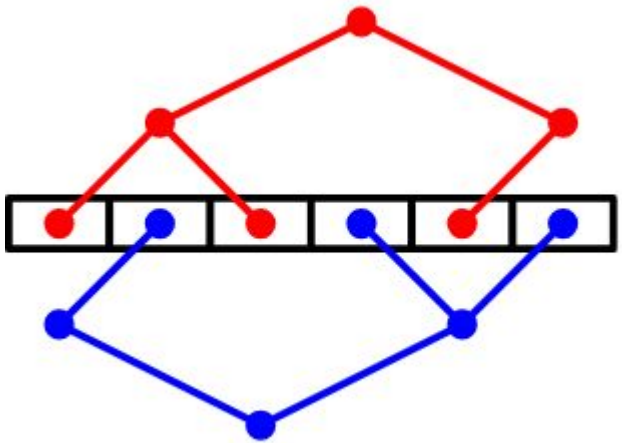
- Use a **binary tree structure** for broadcast
- **Split the message into chunks**
- Forward chunks through the tree in a **pipelined fashion**
- This keeps **many processors active early** while still benefiting tree parallelism

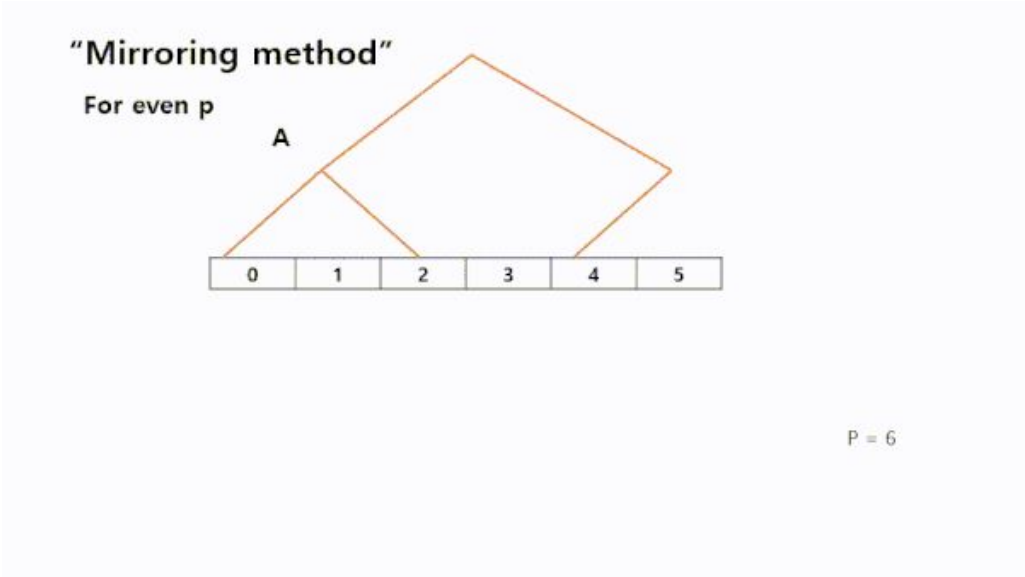


Two Tree

- Two-tree broadcast communicates a message from one root to all other processes using two separate binary trees simultaneously.
- Traditional tree broadcast leaves many nodes as receive-only (leaves).
- Two-tree broadcast uses two trees so that nodes that are leaves in one tree can act as forwarders in the other, increasing utilization.
- The message is split and sent over Tree A and Tree B in parallel, enabling more processes to actively contribute to communication and better using available network bandwidth.

Two Tree Construction - Mirroring





Two Tree - Synchronisation

In two-tree broadcast, **multiple processors communicate in parallel** over **two trees** using alternating steps.

The Problem (if no synchronization is used):

- A processor may try to **send and receive at the same time**
- A processor may receive **two messages in the same step** (from Tree A and Tree B)
- Messages from different steps can **arrive out of order**
- Network links may experience **contention and collisions**

Questions

Q1. Write an MPI program to implement binomial tree broadcast without using `MPI_Bcast`. Execute the program by varying the message size, the number of MPI processes, and the number of nodes, and generate plots showing how the broadcast time varies with each of these parameters.

Q2. Write an MPI program to implement linear pipeline broadcast with message chunking. Run the program for different message sizes, different numbers of processes, and different numbers of nodes, and plot the variation of execution time with respect to message size, process count, and node count.

Q3. Write an MPI program to implement pipelined binary tree broadcast. Evaluate its performance by varying the message size, the number of processes, and the number of nodes, and present plots that illustrate the scalability and performance trends.

Q4. Write an MPI program to implement two-tree broadcast by splitting the message across the two trees and designing your own synchronisation mechanism so that no process sends or receives more than one message in a step. Test the implementation by varying message size, process count, and node count, and plot the resulting performance.

Q5. Compare the performance of all implemented broadcast algorithms with `MPI_Bcast` by varying message size, number of processes, and number of nodes, and present plots that clearly show the differences in scalability and efficiency.

Thank You